Assignment 6 solution

1. 【考点：CPU 运行模式、系统调用、硬件隐指令与内核栈切换】关于操作系统概述中的程序运行环境、系统调用与模式切换，下列叙述中正确的是（ ）

   A. 陷入指令（Trap/Syscall）属于特权指令，只能在内核态（管态）下执行以确保系统安全
   B. 当进程从用户态通过系统调用切换到内核态时，CPU 硬件会自动将用户栈指针切换为内核栈指针，并自动将用户态的断点及处理器状态字压入内核栈中
   C. 系统调用服务例程执行完毕后，内核执行特权指令返回用户态，该指令会自动清空整个 TLB（快表）以防止地址空间泄露
   D. 在系统调用执行期间若发生缺页异常（Fault），操作系统将判定该进程非法并直接将其终止

   【答案】：B
   【解析】：
   A 项错误：陷入指令是用户程序主动申请系统服务时调用的，必须在用户态下执行，若其为特权指令则无法在用户态触发。

   B 项正确：当发生特权级转换时，CPU 硬件隐指令会自动完成栈切换（从 TSS 获取内核栈指针），并将用户态的硬件上下文（PC、PSW、栈指针等）安全压入内核栈中
   C 项错误：返回指令（如 `IRET`）并不一定清空整个 TLB（现代 OS 通常利用 ASID 或仅在切换进程页表基址寄存器时刷新）
   D 项错误：系统调用期间发生缺页属于正常现象，OS 会调页并重新执行访存指令，不会终止进程

2. 【考点：线程实现模型（ULT 与 KLT）、多处理机调度、上下文切换开销】关于进程与线程的实现、多处理机调度及上下文切换，下列说法中错误的是（ ）

   A. 在纯用户级线程（ULT）模型中，若某一个线程因执行阻塞型系统调用而被内核挂起，则整个进程内的所有线程都会被阻塞
   B. 核心级线程（KLT）的调度与切换由操作系统内核直接控制，其上下文切换开销通常大于纯用户级线程
   C. 在对称多处理（SMP）架构下，每个处理机都可以独立运行调度程序并并发访问共享的内存空间
   D. 进程上下文切换时，若切换的两个线程属于同一个进程，则无需重新加载页表基址寄存器（如 CR3）

   【答案】：D
   【解析】：
   A、B、C 项正确：准确描述了用户级线程对内核不可见导致的整体阻塞特性、核心级线程的调度开销以及 SMP 架构的共享内存与独立调度特征
   D 项错误：上下文切换分为进程切换和线程切换。若切换的两个线程属于同一个进程，由于共享虚拟地址空间，确实不需要切换页表基址寄存器；但题目表述为“进程上下文切换时，若切换的两个线程属于同一个进程……”，概念混淆——如果是进程上下文切换（两个不同进程），无论其内部线程如何，都必须切换页表基址寄存器。严格来说，如果是同一个进程内的线程切换，则属于线程级切换而非进程上下文切换

3. 【考点：进程间通信（IPC）、信号机制、信号量与管程特性】操作系统提供了多种进程间通信（IPC）机制与同步互斥工具。下列关于信号、信号量及管道的叙述中，正确的是（ ）

   A. 信号（Signal）是一种异步通信机制，常用于通知进程某个突发事件的发生，它属于低级通信机制
   B. 计数信号量的初值可以被初始化为任意负整数，代表当前正在等待该资源的进程总数
   C. 命名管道（Named Pipe）依托文件系统实现，多个互不相关的进程可以通过管道进行高效的随机读写

D. 管程（Monitor）是一种面向对象的同步机制，允许多个进程同时进入管程内部并发修改共享数据结构

【答案】：**A**
【解析】：
A 项正确：信号是 Linux 等系统中唯一的异步通信机制，属于低级通信工具
B 项错误：信号量的初值绝不能为负数，通常为非负整数
C 项错误：管道本质是先进先出的字节流文件，不支持随机读写（不支持类似 `lseek` 的随机定位）
D 项错误：管程的核心特征是互斥进入，任何时刻最多只能有一个进程在管程内部执行

4. 【考点：死锁预防、死锁避免（银行家算法）、死锁检测与解除】关于死锁的预防、避免与检测，下列叙述中正确的是（ ）

A. 银行家算法属于死锁预防策略，通过在运行动态分配资源时进行安全性检查，确保系统始终处于安全状态
B. 采用“请求和保持条件”的预防策略时，要求进程在开始运行前一次性申请完所需的全部资源，这大大提高了系统资源的利用率
C. 资源分配图（RAG）中若包含环路，则系统中必然已经发生了死锁
D. 死锁检测与解除机制允许系统在运行过程中发生死锁，然后通过周期性地执行检测算法并剥夺资源或终止进程来解除死锁

【答案】：**D**
【解析】：
A 项错误：银行家算法属于死锁避免（Avoidance），而不是预防（Prevention）
B 项错误：一次性分配策略会导致资源严重浪费（许多资源长期闲置）
C 项错误：若资源只有单一实例，环路是死锁的充要条件；若资源有多个实例，环路只是死锁的必要条件而非充分条件
D 项正确：准确描述了死锁检测与解除策略的工作原理

5. 【考点：二级页表地址变换、TLB 命中原理、虚拟内存基本概念】某系统采用请求分页存储管理及二级页表结构，页面大小为 4KB（即 2^{12}B），逻辑地址空间和物理地址空间均为 32 位。页表项大小为 4 字节，且一级页表刚好能够装入一个物理页框中。下列关于该系统地址结构与转换的叙述中，错误的是（ ）

A. 逻辑地址中的页内偏移量占用低 12 位
B. 由于页表项大小为 4 字节，一个 4KB 的物理页框中可以容纳 1024 个页表项，因此二级页表中一级页号和二级页号通常各占 10 位
C. 即使 TLB 命中，由于需要将二级页表项与页内偏移量进行物理拼接，CPU 仍需至少访问一次内存中的二级页表
D. 若未开启 TLB 且采用二级页表，CPU 在每次访存数据前需要依次访问 2 次内存中的页表，最后再访问 1 次内存中的数据，总共需访问 3 次内存

【答案】：**C**
【解析】：
A、B、D 项正确：分别核算了页内偏移量（12位）、页表项划分（10位+10位，刚好占满 4KB 页框）以及二级页表在 TLB 未命中时的 3 次访存开销（2次查页表 + 1次访存）
C 项错误：TLB 命中的核心优势在于硬件直接在高速缓存中命中了物理页框号，根本不需要再去访问内存中的二级页表，直接一步到位获取物理地址

6. 【考点：页面置换算法（FIFO、LRU、Clock）、Belady 异常、系统抖动与工作集】在虚拟存储管理中，关于页面置换算法与系统“抖动（Thrashing）”现象的叙述中，正确的是（ ）

A. FIFO 算法容易出现 Belady 异常（分配页框增加而缺页率上升），而 LRU 算法属于堆栈类算法，绝不会出现 Belady 异常
B. 简单的 Clock 置换算法在寻找淘汰页时，仅检查访问位 U，若 U=1 则将其置 0 并立即淘汰该页
C. 改进型 Clock 置换算法在第一轮扫描时会优先寻找 (U=0, M=1) 的修改页进行淘汰

D. 当系统发生“抖动”时，CPU 利用率下降，此时应当自动增加多道程序度以引入更多进程分担计算负载

【答案】：**A**
【解析】：
A 项正确：LRU 满足栈性质，无 Belady 异常，而 FIFO 会出现
B 项错误：Clock 算法若 U=1 会将其置 0 并跳过（给一次机会），指针前移寻找 U=0 的页，而不是立即淘汰
C 项错误：改进 Clock 优先寻找 (U=0, M=0) 的页面，其次才是 (U=0, M=1)
D 项错误：系统发生抖动时，应当降低多道程序度，减少并发进程数以缓解物理页框争抢

7. 【考点：内存映射文件（mmap）、请求分页、缺页异常与页框回收机制】内存映射文件（Memory-Mapped Files）技术将磁盘文件直接映射到进程的虚拟地址空间中。关于该技术与请求分页、文件管理的融合，下列说法中错误的是（ ）

A. 在执行 `mmap` 系统调用建立映射初期，操作系统仅在进程的虚拟地址空间中创建相应的 VMA，并不实际分配物理页框和拷贝数据
B. 当进程首次访问该映射区域内的虚拟地址时，会触发缺页异常，由操作系统通过缺页处理程序将文件内容调入物理内存
C. 若映射文件对应的某物理页框在内存中被修改（成为脏页），当该页框因内存紧张被淘汰时，系统必须将其写回到外存的交换分区（Swap Space）中
D. 内存映射文件允许多个进程将同一个文件映射到各自的虚拟地址空间中，从而实现高效的共享内存通信

【答案】：**C**
【解析】：
A、B、D 项正确：分别阐述了 mmap 的延迟加载机制、缺页按需调页原理以及多进程共享映射文件的通信机制
C 项错误：内存映射文件的“脏页”背后有具体的磁盘文件实体，淘汰时其修改的数据应直接写回到底层磁盘文件中，而不是写回匿名页专属的交换分区（Swap Space）

8. 【考点：文件元数据、inode、硬链接与软链接的区别、文件删除机制】某文件系统采用 UNIX 风格的索引节点（inode）结构。关于文件元数据、硬链接与软链接的底层机制，下列叙述中正确的是（ ）

A. 为一个普通文件建立硬链接时，会在外存中新分配一个 inode，并让新目录项指向该新 inode
B. 软链接（Symbolic Link）文件拥有自己独立的 inode 和独立的存储数据块，其数据块中存放的是目标文件的路径字符串
C. 当用户通过软链接访问目标文件时，若软链接文件本身的权限为 777，则用户可以无视目标文件的权限限制直接修改目标文件

D. 执行 `unlink` 删除文件时，只要文件的硬链接数归零，操作系统就会立即强行释放该文件的所有内存打开句柄并抹去磁盘数据

【答案】：**B**
【解析】：
A 项错误：硬链接不分配新的 inode，多个目录项共享同一个 inode
B 项正确：软链接是一个独立文件，有自己的 inode 和数据块，里面记录的是目标文件的路径字符串
C 项错误：访问软链接时，VFS 会穿透并最终检查目标文件本身的权限，无法越权
D 项错误：即使硬链接数归零，若内存中仍有进程打开该文件（引用计数大于 0），磁盘数据块也不会被立即回收

9. 【考点：文件物理结构、外存空闲空间管理（位示图）、虚拟文件系统（VFS）】关于文件系统的外存物理结构、空闲空间管理及虚拟文件系统（VFS），下列说法中错误的是（ ）

A. 链接式分配分为显式链接和隐式链接，隐式链接文件虽然不支持高效的随机访问，但完全不会产生外部碎片
B. UNIX 风格的混合索引结构（直接、一级、二级、三级间接指针）能够兼顾小文件的高效访问与大文件的巨大寻址空间
C. 位示图（Bitmap）法常用于外存空闲空间管理，其核心优势在于能够将空闲块管理与磁盘的物理盘块号完美对应，且内存开销极小
D. 虚拟文件系统（VFS）通过定义统一的抽象对象（如 vnode/inode、dentry），使得用户程序在访问不同底层文件系统时必须调用不同的专用系统调用接口

【答案】：D
【解析】：
A、B、C 项正确：分别概括了链接分配的无外部碎片特点、UNIX 混合索引的优势以及位示图的高效性
D 项错误：VFS 的核心目的正是为了提供统一的系统调用接口（如标准的 `open`、`read`、`write`），让上层应用屏蔽底层具体文件系统的差异，而不需要调用不同的专用接口

10. 【考点：I/O 控制方式（DMA）、磁盘调度算法、固态硬盘特性与中断处理】关于输入输出（I/O）管理、外存设备特性及控制方式，下列叙述中正确的是（ ）

A. 在 DMA 控制方式下，数据的输入与输出传输过程完全由 DMA 控制器独立完成，整个传输阶段不需要 CPU 的任何介入
B. 磁盘调度算法（如 SCAN、C-SCAN）的目标是优化平均寻道时间，通常由操作系统内核的设备独立软件层或设备驱动程序实现
C. 固态硬盘（SSD）由于采用闪存芯片，支持原地修改（In-place update）机制，因此其随机写入性能与随机读取性能完全对称
D. 中断处理程序的顶半部（Top Half）可以安全地执行复杂的进程唤醒和阻塞队列调度操作

【答案】：B
【解析】：
A 项错误：DMA 的初始化和结束收尾阶段（响应中断）需要 CPU 介入
B 项正确：磁盘调度算法由操作系统底层设备驱动/独立软件实现，旨在减少平均寻道时间
C 项错误：闪存不支持原地修改，必须先擦除整块才能写入，因此随机写入性能通常远低于随机读性能
D 项错误：顶半部运行在中断上下文中，严禁睡眠或执行复杂的阻塞调度

11. 【考点：CPU 运行模式、系统调用、硬件隐指令与内核栈切换、异常处理】设某 64 位计算机体系结构支持用户态和内核态。当一个处于用户态的进程通过 C 库函数发起 `read` 系统调用时，硬件与操作系统需协同完成一系列微观操作。下列关于该全过程的叙述中，正确的是（ ）

A. 用户程序执行的陷入指令（Trap/Syscall）属于特权指令，必须在内核态下才能被正确识别和执行
B. 发生特权级转换时，CPU 硬件隐指令会自动从当前进程的进程控制块（PCB）中读取内核栈指针，并将用户态的硬件上下文压入内核栈中
C. 若在系统调用服务例程执行期间触发了缺页异常（Fault），操作系统将挂起当前进程并切换至错误处理程序，最终强行终止该进程
D. 系统调用执行完毕后，内核通过执行特权返回指令（如 `sysret` 或 `iret`）恢复硬件上下文，并自动将处理器特权级降回用户态

【答案】：D
【解析】：
A 项错误：陷入指令是用户进程主动发起的，必须在用户态下执行，若其为特权指令则在用户态下无法触发
B 项错误：硬件隐指令通常从 CPU 内部寄存器（如 TSS 或专门的 MSR 寄存器）中获取内核栈指针，而不是直接去实时读取 PCB 结构
C 项错误：系统调用期间发生缺页属于正常现象，操作系统会调页并重新执行引发缺页的指令，不

会直接终止进程
D 项正确：系统调用结束通过特权返回指令一步完成硬件上下文恢复与特权级降级

12. 【考点：用户级线程与核心级线程、SMP 多处理机调度、上下文切换开销】关于操作系统中进程与线程的实现及多处理机调度机制，下列叙述中错误的是（ ）

A. 在纯用户级线程（ULT）模型中，若某个线程因执行阻塞型系统调用而被内核挂起，则整个进程内的所有线程都将陷入阻塞状态
B. 核心级线程（KLT）的调度由内核直接管理，其线程切换需要发生用户态与内核态的模式切换，因此其调度开销显著大于纯用户级线程
C. 在对称多处理（SMP）架构下，多个处理机共享同一主存，且每个处理机都可以独立运行调度程序并竞争就绪队列中的线程
D. 进行进程上下文切换时，若新旧两个线程属于同一个进程，则操作系统可以完全跳过页表基址寄存器（如 CR3）的重新加载

【答案】：D
【解析】：
A 项正确：用户级线程对内核透明，内核只按进程调度，单线程阻塞导致整进程阻塞
B 项正确：KLT 切换需要内核介入，涉及模式切换与硬件上下文保存，开销更大
C 项正确：SMP 架构的核心特征是共享内存与对称的独立调度
D 项错误：如果新旧两个线程属于同一个进程，这属于线程切换（Thread Switch）而非进程上下文切换。在线程切换时确实不需要切换页表基址寄存器；但题目表述为“进行进程上下文切换时，若新旧两个线程属于同一个进程……”，概念自相矛盾——既然是进程上下文切换，就必然意味着切换到了不同的进程，必须重新加载页表

13. 【考点：同步与互斥基本概念、信号量性质、管程特性、原语实现】在操作系统同步与互斥机制中，管程（Monitor）和信号量（Semaphore）是两种核心工具。下列关于它们的叙述中，正确的是（ ）

A. 计数信号量的初值可以被初始化为任意负整数，用于表示当前超额申请该资源的进程数量
B. 管程是一种面向对象的同步机制，它通过共享数据结构和一组过程将数据的定义与操作封装起来，并保证任何时刻最多只有一个进程进入管程内部
C. 当一个进程在管程内部执行 `wait` 操作时，它会释放管程的互斥锁并永久性地退出管程，直到被其他进程显式销毁
D. 信号量的 `P` 操作和 `V` 操作必须作为不可分割的原语执行，且其底层实现绝对不能依赖硬件的中断屏蔽指令

【答案】：B
【解析】：
A 项错误：信号量的初值绝不能为负数，通常为非负整数
B 项正确：管程的核心特征是互斥进入，确保数据安全性与面向对象的封装
C 项错误：管程内部的 `wait` 操作（或条件变量等待）会使进程暂时阻塞并释放互斥锁进入等待队列，而不是永久退出管程，当条件满足时会被 `signal` 唤醒并重新竞争进入管程
D 项错误：原语的底层实现通常必须依赖硬件支持（如开/关中断指令或原子指令如 TSL、XCHG）来保证原子性

14. 【考点：死锁预防、死锁避免（银行家算法）、死锁检测与解除、资源分配图】关于死锁的预防、避免与检测，下列叙述中正确的是（ ）

A. 银行家算法属于死锁预防策略，通过在运行过程中动态实施资源静态分配来破坏“请求和保持”条件
B. 若系统中的资源分配图（RAG）不可约简，则系统中必然已经发生了死锁，且涉及的进程集合即为死锁进程
C. 采用“一次性分配策略”预防死锁时，要求进程在运行前必须一次性申请完整个生命周期中所需的全部资源，这虽然有效但会严重降低资源利用率
D. 死锁检测与解除机制不允许系统在运行过程中发生死锁，而是通过周期性剥夺资源或终止进程来超前阻止死锁

【答案】：C

【解析】：

A 项错误：银行家算法属于死锁避免（Avoidance）而不是预防

B 项错误：若资源有多个实例，资源分配图不可约简（存在环路）只是死锁的必要条件而不是充分条件

C 项正确：一次性分配策略破坏了“请求和保持”条件，能有效防止死锁，但因为许多资源长期闲置而极大地降低了资源利用率

D 项错误：死锁检测与解除属于事后处理策略，允许死锁发生，然后通过检测算法解除，而不是“超前阻止”

15. 【考点：二级页表地址变换、TLB 命中原理、虚拟内存】某 32 位计算机系统采用请求分页存储管理及二级页表结构，页面大小为 4KB（2^{12}B），页表项大小为 4 字节。一级页表和二级页表均恰好占用一个物理页框。下列关于该系统地址结构与转换的叙述中，错误的是（ ）

A. 逻辑地址中的页内偏移量占用低 12 位

B. 由於每个页表占 4KB 且页表项 4 字节，因此页表中包含 1024 个页表项，其逻辑地址中一级页号和二级页号各占 10 位

C. 若 TLB 命中，则 CPU 在进行地址转换时可以直接跳过内存中一级页表和二级页表的查询，直接获取物理页框号

D. 在未开启 TLB 的情况下，CPU 访问一个主存数据单元需要恰好访问 2 次内存（1 次查二级页表，1 次访问数据）

【答案】：D

【解析】：

A、B 项正确：12 位页内偏置对应 4KB 页面；4KB / 4B = 1024 个表项，对应 10 位一级和 10 位二级页号

C 项正确：TLB 缓存了虚拟页号到物理页框号的映射，命中时无需遍历内存页表

D 项错误：在未开启 TLB 且采用二级页表的情况下，CPU 访问一次内存数据需要先访问 1 次内存中的一级页表、再访问 1 次内存中的二级页表，最后才能访问目标数据，总共需要访问 3 次内存，而不是 2 次

16. 【考点：页面置换算法（FIFO、LRU、Clock）、Belady 异常、系统抖动】在虚拟存储管理中，关于页面置换算法与系统“抖动（Thrashing）”现象的叙述中，正确的是（ ）

A. FIFO 页面置换算法可能会出现 Belady 异常，而 LRU 算法由于满足栈性质，因此绝不会出现 Belady 异常

B. 简单的 Clock 置换算法在寻找淘汰页时，若当前指针指向的页面访问位 U=1，则会将其置 0 并立即将其淘汰

C. 改进型 Clock 置换算法在第一轮扫描时，优先选择访问位 U=0 且修改位 M=1 的页面进行淘汰

D. 当系统发生“抖动”时，CPU 利用率急剧下降，此时操作系统应当自动增加多道程序度以引入更多新进程分担计算压力

【答案】：A

【解析】：

A 项正确：LRU 满足堆栈属性，无 Belady 异常；FIFO 存在 Belady 异常

B 项错误：Clock 算法若 U=1 会将其置 0 并跳过（给一次机会），指针前移寻找 U=0 的页，而不是立即淘汰

C 项错误：改进 Clock 算法第一轮优先寻找 (U=0, M=0) 的页面，而不是 (U=0, M=1)

D 项错误：系统发生抖动时，应当降低多道程序度，减少并发进程数以缓解物理页框争抢

17. 【考点：内存映射文件（mmap）、请求分页、页框回收与写回机制】内存映射文件（Memory-Mapped Files）技术将磁盘文件直接映射到进程的虚拟地址空间中。关于该技术与文件系统、内存管理的协同，下列说法中错误的是（ ）

A. 在执行 `mmap` 系统调用建立映射初期，操作系统仅在进程的虚拟地址空间中分配相应的 VMA 区域，并不实际分配物理页框和拷贝数据
B. 进程首次访问该映射区域内的虚拟地址时，会触发缺页异常，由操作系统通过缺页处理程序将文件内容调入物理内存
C. 若映射文件对应的某物理页框在内存中被修改（成为脏页），当该页框因内存紧张被淘汰时，系统必须将其写回到外存的交换分区（Swap Space）中
D. 内存映射文件允许多个进程将同一个文件映射到各自的虚拟地址空间中，从而实现高效的共享内存通信

【答案】：**C**
【解析】：
A、B、D 项正确：分别阐述了 mmap 的延迟加载机制、缺页按需调页原理以及多进程共享映射文件的通信机制
C 项错误：内存映射文件的“脏页”背后有具体的磁盘文件实体，淘汰时其修改的数据应直接写回到底层磁盘文件中，而不是写回匿名页专属的交换分区（Swap Space）

18. 【考点：文件元数据、inode、硬链接与软链接的区别、文件删除机制】关于文件的元数据、硬链接与软链接的底层机制，下列叙述中正确的是（ ）

A. 为一个普通文件建立硬链接时，会在外存中新分配一个 inode 节点，并让新目录项指向该新 inode
B. 软链接（Symbolic Link）文件拥有自己独立的 inode 和独立的数据盘块，其数据块中存放的是目标文件的路径字符串
C. 当用户通过软链接访问目标文件时，若软链接文件本身的权限为 777，则用户可以无视目标文件的权限限制直接修改目标文件
D. 执行 `unlink` 删除文件时，只要文件的硬链接数归零，操作系统就会立即强行释放该文件的所有内存打开句柄并抹去磁盘数据

【答案】：**B**
【解析】：
A 项错误：硬链接不分配新的 inode，多个目录项共享同一个 inode
B 项正确：软链接是一个独立文件，有自己的 inode 和数据块，里面记录的是目标文件的路径字符串
C 项错误：访问软链接时，VFS 会穿透并最终检查目标文件本身的权限，无法越权
D 项错误：即使硬链接数归零，若内存中仍有进程打开该文件（引用计数大于 0），磁盘数据块也不会被立即回收

19. 【考点：文件物理结构、外存空闲空间管理（位示图）、虚拟文件系统（VFS）】关于文件系统的外存物理结构、空闲空间管理及虚拟文件系统（VFS），下列说法中错误的是（ ）

A. 链接式分配分为显式链接和隐式链接，隐式链接文件虽然不支持高效的随机访问，但完全不会产生外部碎片
B. UNIX 风格的混合索引结构（直接、一级、二级、三级间接指针）能够兼顾小文件的高效访问与大文件的巨大寻址空间
C. 位示图（Bitmap）法常用于外存空闲空间管理，其核心优势在于能够将空闲块管理与磁盘的物理盘块号完美对应，且内存开销极小
D. 虚拟文件系统（VFS）通过定义统一的抽象对象（如 vnode/inode、dentry），使得用户程序在访问不同底层文件系统时必须调用不同的专用系统调用接口

【答案】：**D**
【解析】：
A、B、C 项正确：分别概括了链接分配的无外部碎片特点、UNIX 混合索引的优势以及位示图的高效性
D 项错误：VFS 的核心目的正是为了提供统一的系统调用接口（如标准的 `open`、`read`、`write`），让上层应用屏蔽底层具体文件系统的差异，而不需要调用不同的专用接口

20. 【考点：I/O 控制方式（DMA）、磁盘调度算法、固态硬盘特性与中断处理】关于输入输出（I/O）管理、外存设备特性及控制方式，下列叙述中正确的是（ ）

A. 在 DMA 控制方式下，数据的输入与输出传输过程完全由 DMA 控制器独立完成，整个传输阶段不需要 CPU 的任何介入
B. 磁盘调度算法（如 SCAN、C-SCAN）的目标是优化平均寻道时间，通常由操作系统内核的设备独立软件层或设备驱动程序实现

C. 固态硬盘（SSD）由于采用闪存芯片，支持原地修改（In-place update）机制，因此其随机写入性能与随机读取性能完全对称

D. 中断处理程序的顶半部（Top Half）可以安全地执行复杂的进程唤醒和阻塞队列调度操作

【答案】：B
【解析】：
A 项错误：DMA 的初始化和结束收尾阶段（响应中断）需要 CPU 介入
B 项正确：磁盘调度算法由操作系统底层设备驱动/独立软件实现，旨在减少平均寻道时间
C 项错误：闪存不支持原地修改，必须先擦除整块才能写入，因此随机写入性能通常远低于随机读性能
D 项错误：顶半部运行在中断上下文中，严禁睡眠或执行复杂的阻塞调度

21. （1）

```
empty = M   // 表示缓冲区空闲单元数量，初值为 M
full = 0    // 表示缓冲区已有数据的单元数量，初值为 0
mutex_p = 1 // 控制生产者互斥访问缓冲区的信号量，初值为 1
mutex_c = 1 // 控制消费者互斥访问缓冲区的信号量，初值为 1

// 生产者进程
producer() {
    while(1) {
        生产一个数据项;
        P(empty);           // 检查是否有空闲缓冲区
        P(mutex_p);         // 申请对生产端缓冲区的互斥访问
        将数据放入缓冲区;
        V(mutex_p);         // 释放互斥访问权
        V(full);            // 增加满缓冲区计数
    }
}

// 消费者进程
consumer() {
    while(1) {
        P(full);            // 检查是否有可读数据
        P(mutex_c);         // 申请对消费端缓冲区的互斥访问
        从缓冲区取出数据;
        V(mutex_c);         // 释放互斥访问权
        V(empty);           // 增加空闲缓冲区计数
        消费该数据项;
    }
}
```

（2）
安全状态定义：
　如果系统能够按某种进程推进顺序（$P_1, P_2, ..., P_n$）为每个进程分配其所需资源，使得每个进程都能顺利完成，则称系统处于安全状态。安全序列的存在是系统不发生死锁的充分条件
银行家算法安全性检查步骤：
　设置两个向量：工作向量 `Work = Available`（表示系统当前可用的各类资源）；结束向量

`Finish[i] = false`（表示所有进程初始均未完成）

在进程集合中寻找一个满足以下条件的进程 P_i：

`Finish[i] == false`

`Need[i] <= Work`（即进程还需要的资源小于等于当前系统可用资源）

若找到这样一个进程，假设其获得资源顺利运行完毕并释放资源，更新状态：

`Work = Work + Allocation[i]`

`Finish[i] = true`

返回步骤 2 重新循环

若所有进程的 `Finish[i]` 最终均为 `true`，则系统处于安全状态；否则系统处于不安全状态

22. （1）

虚拟地址为 `0x00001024`（二进制：`0000 0000 0000 0000 0001 0000 0010 0100`）

页大小为 4KB = 2^{12} 字节，因此低 12 位为页内偏移量：`0x024`（或十进制 36）

剩余高 20 位用于页号：高 10 位为一级页号（`0`），中间 10 位为二级页号（`1`）

（2）

访存次数：

在不考虑 TLB 命中的二级页表系统中，CPU 访问一次主存数据单元需要访问 3 次内存（第 1 次查一级页表，第 2 次查二级页表，第 3 次访问目标数据单元）

缺页异常微观处理流程：

中断响应与保护现场：硬件 MMU 在地址转换时发现页表项有效位为 0，触发缺页异常（Fault），CPU 自动保存当前进程的硬件上下文（PC、PSW等）到内核栈中，并切换至内核态

转入缺页处理程序：操作系统通过中断向量找到缺页中断服务例程，保护通用寄存器现场

查找辅存与分配物理页：OS 检查该虚拟地址是否合法，若合法则寻找一个空闲物理页框（若无空闲则通过置换算法淘汰页面）

磁盘 I/O 调页：启动磁盘驱动程序，将目标页面从外存交换区调入刚刚分配的物理页框中

修改页表与返回：将页表项中的物理页框号填入并把有效位（Present Bit）置 1。恢复现场，执行特权返回指令（如 `iret`），重新执行引发缺页的那条访存指令

（3）

页面访问序列：1, 2, 1, 3, 1, 4, 2, 3，物理页框数 3

FIFO 算法模拟：

访问 1：内存 `[1]`（缺页）

访问 2：内存 `[1, 2]`（缺页）

访问 1：命中

访问 3：内存 `[1, 2, 3]`（缺页）

访问 1：命中

访问 4：替换最早进入的 1，内存 `[4, 2, 3]`（缺页）

访问 2：命中

访问 3：命中

FIFO 总缺页次数 = 4 次

LRU 算法模拟：

访问 1：内存 `[1]`（缺页）

访问 2：内存 `[1, 2]`（缺页）

访问 1：命中，顺序 `[2, 1]`

访问 3：内存 `[2, 1, 3]`（缺页）

访问 1：命中，顺序 `[2, 3, 1]`

访问 4：替换最久未使用的 2，内存 `[3, 1, 4]`（缺页）

访问 2：替换最久未使用的 3，内存 `[1, 4, 2]`（缺页）

访问 3：替换最久未使用的 4，内存 `[4, 2, 3]` -> 错，此时 LRU 顺序调整为：访问 2 后是 `[1, 4, 2]`，访问 3 替换最久未使用的 1，得到 `[4, 2, 3]`（缺页）

LRU 总缺页次数 = 6 次

23. （1）

盘块大小 4KB = 4096B，每个指针大小 4B

每个磁盘块可容纳的指针数：N = 4096 / 4 = 1024 个

各部分寻址能力：

直接索引：10 × 4KB = 40KB

一级间接索引：1024 × 4KB = 4096KB = 4MB

二级间接索引：1024^2 × 4KB = 1024 × 4MB = 4GB

三级间接索引：1024^3 × 4KB = 4096GB = 4TB

最大文件长度 = 40KB + 4MB + 4GB + 4TB（约等于 4TB）

（2）

总大小：原有 38KB，追加 10KB 后，新文件总大小为 48KB

分析磁盘块分配：

原文件大小 38KB，占用盘块数：⌈ 38KB / 4KB ⌉ = 10 个盘块。恰好将前 10 个直接指针全部占满

新增 10KB 数据，需新增盘块数：⌈ 10KB / 4KB ⌉ = 3 个数据盘块

由于直接索引已满，追加数据必须启用一级间接索引。因此不仅需要分配 3 个数据盘块，还需要额外分配 1 个一级间接索引块

总共需要新分配的磁盘块数 = 3 + 1 = 4 个磁盘块

外存写回次数（最坏情况）：

修改位示图（Bitmap），将分配的 4 个盘块标记为已占用，写回位示图盘块（第 1 次写）

将 3 个新的数据盘块分别写入数据并写回外存（第 3 次写）

将 3 个数据块的物理地址填入一级间接索引块中，写回该一级间接索引块（第 1 次写）

修改内存中的 inode（大小变更为 48KB，且一级间接指针被激活），将 inode 写回外存（第 1 次写）

最坏情况下总写回次数 = 1 + 3 + 1 + 1 = 6 次

24. （1）

本质区别：

中断控制方式：在数据传输阶段，CPU 必须通过执行指令（如轮询或每次传输一个字节/字）不断参与数据的搬运，CPU 的大部分时间被消耗在寄存器与外设端口的读写上

DMA 控制方式：在数据传输阶段，数据的搬运完全由 DMA 控制器（DMAC） 独立控制在内存与 I/O 设备之间进行，整个传输过程不需要 CPU 逐个字节地介入，CPU 可以并发执行其他进程

DMA 结束通知方式：

DMA 传输完成后，DMA 控制器会向 CPU 发送一个硬件中断请求（外中断）

顶半部与底半部的职责区别：

顶半部（Top Half）：运行在严格的中断上下文中，负责对硬件中断进行最紧急的响应：读取状态寄存器、清除中断标志、保存硬件现场，并将底层接收到的数据快速挂入相应队列，随后立即退出，严禁睡眠或执行耗时阻塞操作

底半部（Bottom Half）：推迟执行的任务，负责处理相对耗时的善后工作（如唤醒因等待 I/O 而阻塞的进程、调整进程就绪队列等），允许在软中断或进程上下文中安全执行

（2）

SPOOLing 系统的主要组成部分：

磁盘上的输入井与输出井：在磁盘上开辟的两大存储区域。输入井模拟脱机输入时的磁盘磁带，用于暂存I/O设备输入的数据；输出井用于暂存用户进程要求输出的数据

内存中的输入/输出缓冲区：在内存中开辟的临时存储区，用于缓和磁盘与内存、外设间速度不匹配的矛盾

输入/输出进程（或称预输入/井管理进程、后台守护进程）：在系统运行过程中模拟脱机外围控制机功能的后台进程

解决独占设备冲突的原理：

当多个用户进程同时请求打印输出时，操作系统并没有把真实的物理打印机分配给它们，而是通过 SPOOLing 机制将进程的打印数据直接写入磁盘的输出井中，并为每个进程伪造一台“虚拟打

印机”

宏观上多个进程可以“同时”提交打印任务

真正的物理打印机由后台的输出守护进程独占控制，它按照公平调度原则（如 FIFO）从磁盘输出井中依次读取数据并送到物理打印机上输出，从而彻底解决了独占设备的并发冲突问题